Assignment -3

* Problem Statement

write a C program to create a student database using an array of structures, Apply searching and sorting techniques.

* Objectives

(1) To understand use of an array of structures for maintaining records
(2) To implement and analyze searching techniques.
(3) To implement and analyze searching sorting techniques.
(4) To compose insertion sort, selection sort, shell short.

* Theory

Searching & sorting are two of most fundamental operation in data structures & algorithms. They are widely used in computer application such as database management system, search engines, operating system e-commerce platforms, and information retrieval system.

Efficient searching & sorting techniques improve the performance of software by reducing the time required to locate & organize data.

Linear Search: It is also known as sequential search, it is simplest searching technique
Binary Search: It is efficient searching algorithm that works only on sorted data.
Types of sorting
1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Shell Sort
5. Merge Sort
6. Quick Sort.

* Pseudo code:
write Pseudocode linear search, Binary search, Insertion Sort, Slection & Shell Short.

~~#include <stdio.h>~~
1. Algorithm Linear search (A, n, key)
Start
for i ← 0 to n-1 do
If A[i] = key then
Print "Element found at position", i+1
Stop
End If
End for
print "Element not found"
Step

2. Array must be sorted
Algorithm Binary_search (A, n, key)

Start
low ← 0
high ← n - 1
while low ≤ high do
mid ← (low / high) / 2
if A[mid] = key then
print "Element found at position", mid+1
stop
else if key < A[mid] then
high ← mid - 1
else
low ← mid + 1
End if
End while
Print "Element not ~~found~~ found.
stop.

3 Algorithm Insertion_sort (A, n)
Start
For i ← 1 to n-1 do
key ← A[i]
j ← i - 1
while j ≥ 0 AND A[j] > key do
A[j+1] ← A[j]
j ← j - 1
End while
A[j+1] ← key
End for
Print sorted array
Stop.

4. Algorithm Selection_sort (A, n)
Start
  For i ← 0 to n-2 do
    min ← i
  For j ← i+1 to n-1 do
    If A[j] < A[min] then
    min ← j
  End If
  End for
  Swap A[i] and A[min]
  End for
  Print sorted array
Stop.

5. Algorithm shell_sort (A, n)
Start
  gap ← n/2
  while gap > 0 do
    for i ← gap to n-1 do
    temp ← A[i]
    j - i
    while j ≥ gap AND A[j-gap] > temp do
    A[j] ← A[j-gap]
    j ← j-gap
    End while
    A[j] ← temp
    End for
  gap ← gap/2
  End while
  Print sorted array
Stop.

* Time Complexity
Find out time complexity of above operations.

operations	Best case	Average case	Worst case
Linear Search	O(1)	O(n)	O(n)
Binary Search	O(1)	O(log n)	O(log n)
Insertion sort	O(n)	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Shell sort	O(n log n)*	depends on gap	$O(n^2)$*

* Conclusion
Thus, implemented different searching & sorting methods on the Student database.
This system is able to perform sorting under different cases.

* FAQ's

1. What is the meaning of database & How to maintain in C.

→

A database is an organized collection of data that is stored

and managed so that it can be easily accessed, modified, searched and updated.

How to maintain in C

C does not have built in database system like MySQL or oracle. We can maintain data using:

1. Structures : It is used to store records.
2. Arrays : to store multiple records temporarily.
3. File handling : to permanently store records in files.
4. Functions : for operations such as Add, Search, Update & Delete.

2. What are the applications of Searching ?

→

Searching is process of finding a particular element or record from a collection of data.

1. Database Management
   Finding a particular student, employee, customer, or product record.
2. File Systems
   Searching for specific file or folder.
3. Information Retrieval
   Finding information from

large collection of documents or websites.

(4) Banking Systems
Searching customer accounts & transaction records.

(5) E commerce
Finding products based on product name, category or ID.

(6) Hospital Management
Searching patient records and medical information.

3. Compose linear search & binary search.

→

Basis	Linear search	Binary search
Method	Checks elements one by one	Divides the array into halves
Data requirement	Array need not be sorted	Array must be sorted
Best case	O(1)	O(1)
Average case	O(n)	O(log n)
worst case	O(n)	O(log n)
speed	Slower for large data	Faster for large sorted data
Implementation	Simple	Slightly more complex

4. What are the applications of sorting?

→

(1) Searching:
Sorting makes binary search possible & efficient.

(2) Student record
Arranging students according to roll no, marks, or name.

(3) Employee Management
Sorting employees according to salary, ID or designation.

(4) Data Analysis
Sorting data helps in finding minimum, maximum, median, and trends.

(5) Ranking System
Arranging players or students according to scores or performance.

(6) File management
Arranging files according to name, size, type or date.

5. Compare and Contrast, selection, insertion sort and shell sort.

→

Basis	Selection	Insertion	Shell
Basic idea	Selects the smallest element & places it in correct position	Takes one element and insert it into the sorted part.	Perform insertion sort on elements separated by gap

Best case	$O(n^2)$	$O(n)$	Depend on gap
Average case	$O(n^2)$	$O(n^2)$	Depend on gap
worst case	$O(n^2)$	$O(n^2)$	Depend on gap
Space complexity	$O(1)$	$O(1)$	$O(1)$
Stable	No	Yes	Generally No
Speed	Slow	Fast	Generally fast

6. What ~~are~~ is external & internal sorting, In place, Stable features of sorting Algorithm.

→

① Internal Sorting

Internal sorting is a sorting technique in which all the data to be sorted fits into the main memory that is (RAM).

② External Sorting

External sorting is used when the data is too large to fit into main memory. Data is stored partly on secondary storage such as a hard disk or SSD.

③ Inplace Sorting

An Inplace sorting algorithm sorts the data using very little extra memory, usually $O(1)$ auxiliary space.

C Program:

```
#include <stdio.h>

struct Student
{
    int rollNo;
    char name[50];
    float marks;
};

void inputStudents(struct Student s[], int n)
{
    int i;

    for (i = 0; i < n; i++)
    {
        printf("\nEnter details of student %d\n", i + 1);

        printf("Roll No.: ");
        scanf("%d", &s[i].rollNo);

        printf("Name: ");
        scanf(" %[^\n]", s[i].name);

        printf("Marks: ");
        scanf("%f", &s[i].marks);
    }
}

void displayStudents(struct Student s[], int n)
{
    int i;

    printf("\nRoll No.\tName\t\tMarks\n");
    printf("----------------------------------------\n");

    for (i = 0; i < n; i++)
    {
        printf("%d\t\t%s\t\t%.2f\n",
               s[i].rollNo, s[i].name, s[i].marks);
    }
}

void searchStudent(struct Student s[], int n)
{
    int roll, i, found = 0;

    printf("\nEnter roll number to search: ");
    scanf("%d", &roll);

    for (i = 0; i < n; i++)
    {
        if (s[i].rollNo == roll)
        {
```

```
            printf("\nStudent Found\n");
            printf("Roll No. : %d\n", s[i].rollNo);
            printf("Name     : %s\n", s[i].name);
            printf("Marks    : %.2f\n", s[i].marks);
            found = 1;
            break;
        }
    }

    if (found == 0)
        printf("\nStudent not found.\n");
}

void sortStudents(struct Student s[], int n)
{
    int i, j;
    struct Student temp;

    for (i = 0; i < n - 1; i++)
    {
        for (j = 0; j < n - i - 1; j++)
        {
            if (s[j].rollNo > s[j + 1].rollNo)
            {
                temp = s[j];
                s[j] = s[j + 1];
                s[j + 1] = temp;
            }
        }
    }

    printf("\nStudents sorted by roll number.\n");
}

int main()
{
    int n;

    printf("Enter number of students: ");
    scanf("%d", &n);

    struct Student students[n];

    inputStudents(students, n);

    printf("\nStudent Database:\n");
    displayStudents(students, n);

    searchStudent(students, n);

    sortStudents(students, n);

    printf("\nSorted Student Database:\n");
    displayStudents(students, n);
```

```
    return 0;
}
```

Output:

```
Enter number of students: 4

Enter details of student 1
Roll No.: 1
Name: Eshan
Marks: 92

Enter details of student 2
Roll No.: 2
Name: Archit
Marks: 94

Enter details of student 3
Roll No.: 3
Name: Aarushi
Marks: 97

Enter details of student 4
Roll No.: 4
Name: Anushka
Marks: 97

Student Database:

Roll No.        Name            Marks
---------------------------------------
1               Eshan           92.00
2               Archit          94.00
3               Aarushi         97.00
4               Anushka         97.00

Enter roll number to search: 3

Student Found
Roll No. : 3
Name     : Aarushi
Marks    : 97.00

Students sorted by roll number.

Sorted Student Database:

Roll No.        Name            Marks
---------------------------------------
1               Eshan           92.00
2               Archit          94.00
3               Aarushi         97.00
4               Anushka         97.00
```